刘鑫海

电子简历网站：https://liuxinhai.tech （有更详细的履历信息）

28岁 | 5年经验 | 17729500571 | 741183972@qq.com



Personal resume

教育背景

2021.6-2014.6 **西安邮电大学** **网络工程（本科）**

大学实习：2019.8 滴滴网约车 ， 2019.11 小米官网， 2020.3 腾讯直播，2020.6百度凤巢，2020.9 QQ浏览器
学业：连续两年专业第一，英语六级， 荣誉称号若干（校三好学生，优秀班长，优秀党员...）

工作经验

2025.8-至今 **京东・京ME・JoySpace文档** **WEB开发**

技术栈： React + TypeScript
业务：负责「Joyspace文档」的需求评审和落地
技术：推动 AI 相关「业务能力」建设

2022.8-2025.8 **字节・番茄小说** **跨端开发**

技术栈： React（PIA/Lynx）+ TypeScript + Java（Kotlin）
业务：故事社区 owner，主导KA需求的技术方案与落地。参与番茄小说管理后台开发。
技术：跨端建设专项 owner，主导落地 Lynx TTML Nodiff、ReactLynx3、PIA。参与大前端（安卓）开发，主要是跨端容器。

2022.8-2017.7 **腾讯・QQ浏览器・小说业务线** **跨端开发**

技术栈： React（Hippy） + Vue + Koa + TypeScript
业务：负责小说福利tab的需求评审和落地。参与小说阅读器、听书、小说频道、UGC 社区的开发。
技术：参与团队 npm 管理方案、适老化改造、飞鱼低代码平台共建。

项目经验

2026.5-2026.5 **Joyspace 文档智能创作**

背景：让用户在文档编辑器内完成「查 → 写 → 改」闭环，把 AI 从「被动改写工具」升级为「伴随式写作伙伴」。
设计：开发文档Web MCP，抽离通用 Agent SDK，完成「AI决策」harness工程。
收益：实验已全量，服务于京东全员，降低长文写作门槛

2026.9-2026.9 **AITeamSpace**

背景：PRD / 接口 / 方案 / 用例散落云文档，Agent 读不到，只能截图粘贴；改验收标准靠群里人肉同步。AI 提了单兵效率，没解决「协作上下文怎么给 AI」。
设计：在「共享云文档 / 内部知识库平台 / Git 仓库」三条路线中选 Git——AI 可直接读 md、原生 diff 与版本、能挂进 Agent Workspace。 设计并落地 Git仓库 的规约，让AI可读、可写、可维护一套需求的完整上下文。
收益： 需求的流程化输入输出，人、AI 均可读、可维护， 上下文更新会机器人群通知

2024.8-2025.3 **RL3 跨端框架迁移专项**

背景：RL2 只能写 Class、开发体验差、双端 bug 多；看到抖音搜索业务迁 RL3 后 FMP 明显提升，我判断这条路线值得跟进。主动调研内部 RL3 落地数据、评估迁移成本后发起专项，从「要不要迁」到「怎么迁不翻车」全程负责，推动消息中心、书荒广场等核心页落地。
核心动作：
（1）迁移策略：制定分批迁移 Roadmap（存量 14 页 + 新增 7 页）、灰度实验方案与回滚策略；搭 Slardar 数据看板 + FMP/错误率异常报警，迁移期间 0 事故、0 回滚。
（2）工程基建：针对迁移中暴露的共性问题自研插件链——CssVariableTreeShaking 删冗余颜色变量（单页 −20 KB）；ESLint 插件自动检测并一键修复可优化写法，控制产物体积膨胀。
（3）团队沉淀：统一 RL3 开发规范，沉淀 SwiperAction / ShowMonitor 等公共组件与日夜间、大字号适配方案；做 VS Code 代码模板插件，单页开发节省 0.5pd（后续页面可一键生成，不再重复踩坑。）

2024.1-2025.8 **短故事社区**

背景：短故事赛道约 4700 万 DAU 空间，故事社区作为番茄 LT30 增长抓手从 0 搭建，首屏性能是生死线。
设计：从 RN、Webview、PIA、Lynx 四条跨端路线中，主导技术选型规范：首屏敏感页优先 Lynx（TTML Nodiff / ReactLynx 3.0），复杂交互页收敛到 React + Arco；推动 PIA 生态在书圈、帖子详情页、机器人详情页落地，最终形成「页面用 RL3、页卡用 Nodiff、丰富交互 PIA」的团队级技术规范。
核心动作：
（1）书圈落地页低端机性能治理：围绕低端机首屏体验，深入分析后，采用「请求瘦身、渲染优化」优化，安卓机 PCT90 从 5.28s 降至 3.8s。
（2）PIA 书圈：通过 PIA worker 优化，FCP 从 4s 降至 2.1s。
（3）帖子详情页秒开：设计「客户端数据注入 + 伪 SSR」。伪 SSR chunk 前置拆分，绕开 React 运行时直出首屏，FCP 从 1.5s 压到 160ms。
（4）夜间模式IOS闪白：排查到是IOS内核对defer兼容性问题，开发Webpack插件进行bundle拆分解决。

2021.6-2022.8 **QQ浏览器 福利tab**

背景：福利中心强依赖终端首屏表现，接入后发现首屏有效率只有 41%——一半用户等不到有效内容就走了。2360ms 的首屏耗时里，前端、WebView 容器、网络请求各占一段，单靠业务页面前端自己搞不定。
核心动作：
（1）数据定位：用直出数据做负反馈纠偏，把首屏耗时拆到容器启动 / 白屏 / 请求 / 渲染四段，定位到瓶颈不只在业务代码。
（2）跨端协同：主动拉 WebView 容器侧联合优化（容器预加载 / 预热），把容器启动这段时间从链路里压掉。
（3）前端侧治理：首屏请求管制（非首屏请求延后）+ 骨架屏直出，白屏期间有内容占位而不是干等。

技能

跨端与框架：熟练 React、Vue、TypeScript，了解相关源码实现。
基础与工程：熟悉浏览器、HTTP、Webpack构建工具
服务端：了解Node开发，在腾讯时开发过Koa，了解安卓客户端开发，在字节时开发过容器层

自我评价

5年前端经验，深耕跨端与首屏性能。主导过技术专项（ RL3 迁移、PIA 秒开等），擅长从数据定位问题、用工程手段把性能提升到极致。
不满足于只写业务代码： 做过安卓开发和服务端开发，关注 AI 编码协作方向，致力于AI高效协同办公。
有产品 sense，会主动判断 "该不该做、值不值得做"，并从技术角度赋能业务。